

Verification Continuum™

Verdi® nECO User Guide and Tutorial

Version W-2024.09, September 2024



Copyright and Proprietary Information Notice

© 2024 Synopsys, Inc. This Synopsys software and all associated documentation are proprietary to Synopsys, Inc. and may only be used pursuant to the terms and conditions of a written license agreement with Synopsys, Inc. All other use, reproduction, modification, or distribution of the Synopsys software or the associated documentation is strictly prohibited.

Destination Control Statement

All technical data contained in this publication is subject to the export control laws of the United States of America. Disclosure to nationals of other countries contrary to United States law is prohibited. It is the reader's responsibility to determine the applicable regulations and to comply with them.

Disclaimer

SYNOPSYS, INC., AND ITS LICENSORS MAKE NO WARRANTY OF ANY KIND, EXPRESS OR IMPLIED, WITH REGARD TO THIS MATERIAL, INCLUDING, BUT NOT LIMITED TO, THE IMPLIED WARRANTIES OF MERCHANTABILITY AND FITNESS FOR A PARTICULAR PURPOSE.

Trademarks

Synopsys and certain Synopsys product names are trademarks of Synopsys, as set forth at <https://www.synopsys.com/company/legal/trademarks-brands.html>. All other product or company names may be trademarks of their respective owners.

Free and Open-Source Licensing Notices

If applicable, Free and Open-Source Software (FOSS) licensing notices are available in the product installation.

Third-Party Links

Any links to third-party websites included in this document are for your convenience only. Synopsys does not endorse and is not responsible for such websites and their practices, including privacy practices, availability, and content.

www.synopsys.com

Contents

Audience	6
Customer Support	6
Synopsys Statement on Inclusivity and Diversity	7

1. Introduction	8
Overview	8
Terminology	9
Dedicated ECO Window	10
Modify Connection in ECO Window	10
Make Connection	11
Connect to Net	11
Delete Connection	11
Cell Modification	11
Add Instance	11
Delete Instance	12
Replace Instance	12
Undo/Redo	12
Commit Change	12
Freeze Silicon Mode	12
Add Instance	13
Delete Instance	13
Replace Instance	13
ECO Utilities	13
Clone Module	13
Change Instance Scope	13
Write the New Netlist to New HDL Files	14
Steps to Use the nECO Module	14

2. User Interface	16
ECO Window	16
Frame Banner	16

Pull-down Menu	17
Mnemonic Key	17
Bind Key	17
Toolbar	17
Mouse Operation	17
nECO Toolbar Commands	18
nECO Mouse Operations	18
3. Application Tutorials	20
Before You Begin	20
Installation and Setup	21
Non-Freeze Silicon ECO	21
Import a Verilog Gate Design	21
Isolate the Logic for Modification	22
Check Hierarchies and Insert Buffers	23
Insert an Inverter between Instances	23
Change Placement and Routing	26
Commit Change	27
Review Source Code	28
Save ECO Results to a File	29
Freeze Silicon ECO	30
Import a Verilog Design	31
Manage Spare Cell by Instance Name	31
Manage Spare Cell by Cell Number	39
Clone a Module	46
Import Your Design	46
Check the Preferences Setup	46
Isolate the Logic for Modification	47
Clone Module	47
Change Instance Scope	51
Import Your Design	51
Change Instance Scope	51
Save HDL Code to a File	54
4. Appendix A: Novas ECO Script	55
Commands and Key Components	55

Contents

Script File Format 56

Loading the ECO Script Through Batch Mode62

Example62

Preface

This book is designed to allow you to quickly become proficient in the *nECO* module, an add-on to the Verdi platform. You should already be familiar with the Verdi platform (in particular *nSchema*) before beginning this book. If you do not feel comfortable with basic Verdi operations, review the *Verdi User Guide and Tutorial* document first.

This book focuses on the most commonly used commands without going into detail on everything. For detailed descriptions of individual commands, refer to the appropriate chapter of the *Verdi and Siloti Command Reference Manual*.

The manual should be read from beginning to end, although you may skip any sections which you are already familiar.

Audience

The audience for this manual includes engineers who are familiar with languages and tools used in design and verification and are frequently required to complete Engineering Change Orders (ECOs) on gate-level netlists.

This document assumes that you have a basic knowledge of the platform on which your version of the Verdi platform runs: UNIX or Linux and that you are knowledgeable in Verilog or VHDL, simulation software, and digital logic design.

Customer Support

For any online access to the self-help resources, you can refer to the documentation and searchable knowledge base available in SolvNetPlus.

To obtain support for your Verdi product, choose one of the following:

- Open a case through SolvNetPlus.

Go to <https://solvnetplus.synopsys.com/s/contactsupport> and provide the requested information, including:

- Product L1 as Verdi
- Case Type

Fill in the remaining fields according to your environment and issue.

- Send an e-mail message to verdi_support@synopsys.com.

Include product name (L1), sub-product name/technology (L2), and product version in your e-mail, so it can be routed correctly.

Your e-mail will be acknowledged by automatic reply and assigned a Case number along with Case reference ID in the subject (ref:____:ref).

For any further communication on this Case via e-mail, send e-mail to verdi_support@synopsys.com and ensure to have the same Case ref ID in the subject header or else it will open duplicate cases.

Note:

In general, we need to be able to reproduce the problem in order to fix it, so a simple model demonstrating the error is the most effective way for us to identify the bug. If that is not possible, then provide a detailed explanation of the problem along with complete error and corresponding code, if any/permissible.

Synopsys Statement on Inclusivity and Diversity

Synopsys is committed to creating an inclusive environment where every employee, customer, and partner feels welcomed. We are reviewing and removing exclusionary language from our products and supporting customer-facing collateral. Our effort also includes internal initiatives to remove biased language from our engineering and working environment, including terms that are embedded in our software and IPs. At the same time, we are working to ensure that our web content and software applications are usable to people of varying abilities. You may still find examples of non-inclusive language in our software or documentation as our IPs implement industry-standard specifications that are currently under review to remove exclusionary language.

1

Introduction

This section describes the following topics:

- [Overview](#)
- [Dedicated ECO Window](#)
- [Modify Connection in ECO Window](#)
- [Cell Modification](#)
- [Commit Change](#)
- [Freeze Silicon Mode](#)
- [ECO Utilities](#)
- [Write the New Netlist to New HDL Files](#)
- [Steps to Use the nECO Module](#)

Overview

Designers of complex chips find many reasons to modify the netlist. Often these changes come late in the design process where errors are very costly. Netlist changes are error-prone because the netlist is large, complex, unfamiliar, and hard to understand. Errors such as changing the wrong gate or making a typo are common. The *nECO* module is designed to solve this problem. It helps you to isolate the logic that needs to change in an ECO window so you can easily modify the design. After the modification, you can commit this change and *nTrace* and *nSchema* reflects the changes immediately. When you export the changes to new HDL source files, comments are automatically added to the new HDL files to keep the ECO history.

The *nECO* module is an additional component available with the Verdi platform and embedded within the Verdi platform. Its use model is exactly the same with other components of the Verdi platform.

The *nECO* module provides the following capability:

- Dedicated ECO window to isolate logic that needs to be changed.
- Modify the connection between logic gates.
- Add/Delete logic gates.
- Undo/Redo.
- Commit the modification to reflect the change to source code and schematic.
- Write the modified netlist to new HDL files.
- Automatically add comments to the source code to keep the ECO change history.
- ECO utilities to create ports, change cell type, clone module, and move instances to a different scope.
- Support freeze silicon mode and non-freeze silicon mode.
- Support Novas ECO script in Tcl and Avant! ECO script format for the change.
- Add/Delete buffers.
- Rename nets or instances.
- Manage the spare cells by cell number or instance name.

Terminology

ECO Window

This is the dedicated window for you to modify your netlist.

Library Cell

This is the most basic module in gate level netlist and is defined in the Verdi platform as a symbol library.

Cell and Component

Instances of library cell.

Instance Pin

The pins of a component.

Dedicated ECO Window

The ECO window is the main window of the *nECO* module. This window displays the circuit in a flattened schematic style to isolate logic from other non-related circuits. You can modify the design in this window. Any modification made in this window does not take effect until the **Commit Change** command is invoked. After the **Commit Change** command, the internal database of the Verdi platform is updated and the content in *nTrace* and *nSchema* reflects the change immediately. You can verify whether the changes meet the requirement.

Following are the two ways to invoke the ECO window.

1. You can select the components you want to modify in any *nSchema* window and invoke the **Tools -> New Schematic -> ECO Window** command. The ECO window opens and displays the selected components exactly the same as the display style in the flattened window.
2. You can collect all the interesting components in a flattened window first and then invoke the **Tools -> New Schematic -> ECO Window for All** command. The ECO window opens and displays exactly the same copy of the content in the flattened window.

The ECO window acts like scratch paper. Any changes made in the ECO window does not affect the real design until you commit the modification. Hence it is safe and local. Also if you miss any components when you created the ECO window, you can incrementally drag and drop them from other Verdi windows.

To avoid confusion, only one ECO window is allowed in the Verdi platform. If the ECO window is opened, the commands to invoke ECO window are disabled.

Modify Connection in ECO Window

The instance pin is the basic unit to make changes in the ECO window. All the connection modification actions are related to the instance pin. Also the connection status of the instance pin is very important. You cannot connect a connected instance pin to another net. An Xmark is displayed next to the instance pin if this instance pin is floating (not connected). Also the **View -> Fan In/Out Number** command turns on the display of the fan in/out number to provide more detailed connection information.

ECO provides the following ways to modify the connection:

Make Connection

This command is used to make a connection between two objects. The connection can be between:

- instance pin - instance pin
- instance pin - net

Note:

In both cases, at least one of the instance pins must be floating.

This command is very flexible, you can:

- Select two objects and invoke this command.
- Select one object, invoke this command, and then select another object.

Connect to Net

This command is used to connect the selected instance pin to the specified net. This is a short cut to the make connection command.

Delete Connection

The delete connection command can be applied to an instance pin or a net. If applied to a net, all the instance pins are disconnected from this net. If applied to an instance pin, the instance pin are disconnected from the net it is connected.

Cell Modification

In the ECO window, you can add cells to the design, delete cells from the design, or replace a cell with a different type. The following commands facilitate cell modification.

Add Instance

This command allows you to add an instance from the cell library to this design. While adding an instance, you must specify the scope you want to add the instance into. This command lists all the scopes currently referenced in the ECO window for you to choose from or you can type in another scope.

Delete Instance

This command deletes the selected instances from the design.

Replace Instance

This command allows you to replace the selected instance with another cell type. The form lists the possible candidates according to the pin name match result. You can choose to list those cells with exactly the same pin name and pin count or those cells that have pins that cover the selected cell.

If the **Freeze Silicon Mode** is turned *on*, the behavior of these commands are different. It deals with the cells in the spare cell list.

Undo/Redo

Multiple levels of **Undo/Redo** are provided for you to undo/redo the action in the ECO window.

Commit Change

All the editing you do in the ECO window does not take effect until you invoke the **Commit Change** command. This command checks the connection and cell status in the current ECO window and gives you a report to verify. Also it displays the default comment that is added to the source code with date (by default) for this change. You can change it to your preferred comment. Once you commit the form, it updates the changes to Verdi internal knowledge database and you see the changes in other windows such as *nTrace* and *nSchema*.

Freeze Silicon Mode

The *nECO* module provides **Freeze Silicon Mode** for the post layout ECO stage. In this mode, the cells in the design are fixed. All the changes only impact the connection.

To set this mode, you need to turn on the **Freeze Silicon** option on the **Tools -> Preferences -> Schematic** page -> **ECO** page -> **Freeze Silicon** page. When this option is turned *on*, you can set the other options for the spare cell rules in the design.

In **Freeze Silicon Mode**, all the cell modification commands deal with the cells in the spare cell list. The **Edit -> Spare Cell** command in the ECO window is for you to create the spare cell list. You can load the spare cell list from a file that may be prepared by you or saved from a previous *nECO* session. You can also use naming rule matching to search

the spare cells. The content in the spare cell list varies with the modifications in the ECO session. Also, you can save the final spare cell list to a file for future retrieval.

The biggest difference in **Freeze Silicon Mode** is that the cell modification commands does not modify the cells in the design. Instead, they are dealing with the spare cell list.

Add Instance

This command lists all the instances available in the spare list for you to add. When an instance is added from the spare list, this instance is deleted from the spare list.

Delete Instance

This command adds the deleted instance to the spare cell list.

Replace Instance

This command only lists the cells in the spare cell list with the matched pin status. When you replace the selected cell with another cell in the spare cell list, the replaced instance is added to the spare cell list and the chosen cell is removed from the spare cell list.

The spare cell rule is applied to the new added spare cells.

ECO Utilities

In addition to the ECO window, the *nECO* module also provides some ECO utilities for you to change your design hierarchy. Following are two utilities available:

Clone Module

A module may have several instantiations in the design. In some scenarios, you only want to make a change to one or some instances of this module. For this case, you need to clone these instances into different modules to make them distinct. Giving the module name, this command lists all the instances for this module in this design and then you can clone these instances according to your need.

Change Instance Scope

This command is for you to move an instance from one scope to another scope. You select an instance in the schematic first then invoke **Tools -> ECO Utility -> Change Instance Scope** command. While moving this instance to the specified scope, this

command automatically handles the connection and hierarchical port modification from this change.

Write the New Netlist to New HDL Files

When you have finished all the modifications by using the ECO window or an ECO utility, you can write the new netlist to HDL files by choosing the **File -> ECO -> Save ECO Netlist** command in the *nTrace* window. You can choose to save only those HDL files that are changed in this ECO process or you can save all the HDL files starting from a scope. This command does not override your original HDL files. Instead, it writes all the new HDL files under the specified directory and keeps the original file names.

The content in the saved HDL files is the same as your original HDL files except for what is modified. The comment specified when committing the change is added to the modified part in the new HDL files. Also, you can use the **File -> ECO -> ECO Report** command to generate a report for all the changes you made during this ECO session.

This feature supports both **Freeze Silicon Mode** and **non-Freeze Silicon Mode**.

Steps to Use the nECO Module

Perform the following steps to use the nECO module:

1. Check whether your Verdi library is prepared and whether the *NOVAS_LIBS* and *NOVAS_LIBPATHS* environment variables are set correctly.

Note:

The **-ssz**, **-ssy** and **-ssv** options of *vericom* or *verdi* must NOT be used when compiling the gate-level design in nECO.

2. Invoke the Verdi platform to import your design.
3. Invoke *nSchema* to display the schematic.
4. If the design is in freeze silicon status:
 - Enable the **Freeze Silicon** option on the **Tools -> Preferences -> Schematics** page-> **ECO** page -> **Freeze Silicon** page.
 - Invoke the **Tools -> ECO Utility -> Spare Cell** command in *nSchema* to prepare your spare cell list.
5. Select the components you want to change in the schematic window and invoke the *nECO* command to bring up the ECO window.

6. An alternative to step 5 is to collect all the components in a flattened window first and invoke the ECO window for all command.
7. In the ECO window, modify your netlist.
8. After you have made all the changes, invoke the **Commit Change** command. You find the changes in both *nTrace* and *nSchema* windows.
9. Close this ECO window.
10. You can repeat steps 5 to step 9 for all the changes you want to make.
11. Invoke the **File -> ECO -> Save ECO Netlist** command to save the new HDL files.
12. In **Freeze Silicon Mode**, invoke the **Tools -> ECO Utility -> Spare Cell** command to write the final spare cell list to a file for a future ECO session.
13. Use the **File -> ECO -> ECO Report** command to generate a report about this ECO session.
14. Exit the Verdi platform.

2

User Interface

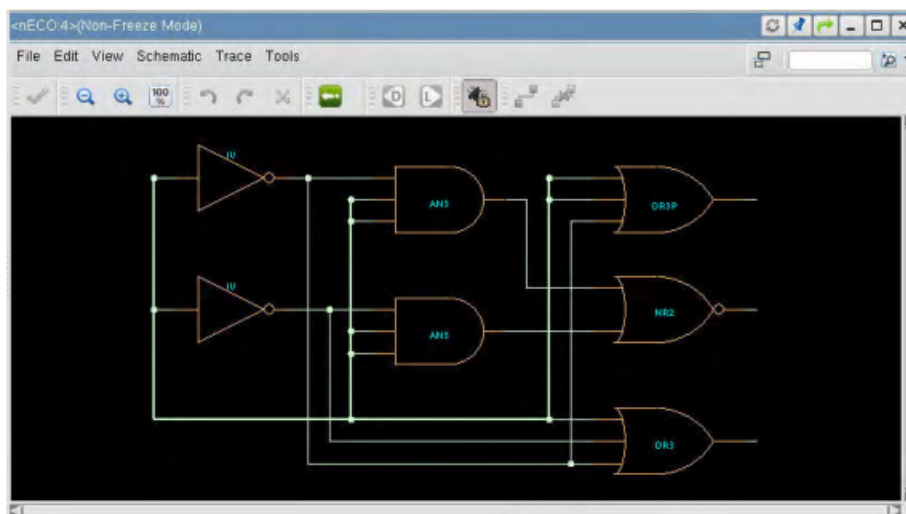
This section describes the following topics:

- [ECO Window](#)
- [nECO Toolbar Commands](#)
- [nECO Mouse Operations](#)

ECO Window

After you invoke *nSchema* for ECO modification, the *nECO* frame is displayed as illustrated in the following figure:

Figure 1 *nECO* Frame



Frame Banner

The banner at the top of each *nECO* frame identifies the application and frame number (such as <nECO:4>) and the operation mode (Non-Freeze Mode or Freeze Mode).

Pull-down Menu

A pull-down menu bar is located just below the frame banner. Each menu item contains several commands that display when the menu is selected. The *nECO* module supports the tear-off menu convention. This allows you to pull aside a frequently used menu on Unix platforms and leave it open.

Mnemonic Key

The pull-down menus support **Meta** key invocation using mnemonics. The mnemonic for each item is indicated by an underline. For example, to display the **File -> Open** menu (meta -fo), click and hold the <Meta> key on the keyboard (the diamond key/<Alt> key on Sun keyboards or the <Alt> key on Windows keyboards) and click the "f" key, then release the <Meta> key and click the letter "o" key.

Bind Key

You can bind a command to either a keystroke or a mouse button. Once you have defined bind keys, you can easily invoke commands with a keystroke or mouse click. For example, the **Source -> Active Annotation** command can be invoked using the "x" key (the defined bind key is the letter after the command in the menu).

Toolbar

A row of icons appears below the pull-down menu bar. These icons give you access to frequently used commands for the current window.

Mouse Operation

The mouse is most often used to select objects by clicking the left mouse button. You can select ranges of objects by dragging with the left button over the objects. You can also select a range by using the <Shift> key along with clicking-left. To add or remove individual objects to or from the selection, use the <Ctrl> key along with click-left.

The *nECO* module also makes use of drag-and-drop operation to move information from one frame to another. You normally perform drag-and-drop by pressing the middle mouse button to select the object, holding the button as you move the mouse to a new location, then releasing the button to "drop" the object into a new location.

nECO Toolbar Commands

A row of icons appears below the pull-down menu bar. These icons provide access to frequently used commands for the current window.

The available toolbar icons may be modified using one of the following methods:

- Enable/disable the icon category using the right-click menu option on the main window.
- Left-click to select the separator bar and then drag left or right to decrease or increase the associated space. When the space is decreased such that some icons are no longer visible, a double arrow (>>) symbol is displayed to the right of the category. Click this symbol to display the hidden icons.
- Left-click and select the gray bar, then drag up or down to undock the category and then move to a new location on the toolbar or the left/right/below of the window. Available slots are highlighted with a blue dashed line.
- Define/modify/add toolbar icons and categories using the **Tools -> Customize Menu/Toolbar** command.

Refer to the *nECO* chapter of the *Verdi and Siloti Command Reference* manual for details on the available toolbar icons.

nECO Mouse Operations

The following table lists the *nECO* mouse actions.

Mouse Action	Schematic Window
Left-click a signal/instance	Deselect the previous selection and select the current signal/instance.
Shift-left-click a signal/instance	Add the signal/instance to a selection list for multiple signals/instances selection.
Left-click anywhere without a signal/instance	Deselect all.
Drag-left	Zoom in an area.
Right-click	Open a context-sensitive menu.
Double-click a signal	Trace the signal's drivers by original connections only.
Drag & Drop an instance to a waveform window	Display the waveforms for the corresponding instance's I/O signals.

Drag & Drop a signal to a waveform window	Display the corresponding signal's waveform.
Drag & Drop an instance to a source code frame	Find and highlight the associated instance in the source code pane.
Drag & Drop an instance block to a Temporal Flow View frame	Highlight the corresponding instance's output signal if it exists. Add the instance as a reference if it does not exist. The global cursor time is used to identify the signal.
Drag & Drop a signal to a Temporal Flow View frame	Highlight the corresponding signal if it exists. Add the signal and driving instance as a reference if it does not exist. The global cursor time is used to identify the signal.
Drag & Drop a signal to a source code frame	Trace the connectivity of the signal in the source code pane.
Ctrl + Drag-left	Pan the <i>nECO</i> window.

3

Application Tutorials

This chapter describes the basic functionality of the *nECO* module. As an example, the tutorial uses a microprogrammed CPU design with gate-level source.

This chapter contains the following sections:

- [Before You Begin](#)
- [Non-Freeze Silicon ECO](#)
- [Freeze Silicon ECO](#)
- [Clone a Module](#)
- [Change Instance Scope](#)

Before You Begin

Before you begin the tutorials, you (or your system manager) must have installed the Verdi and Siloti systems as described in the accompanying *Installation and System Administration Guide*.

Note:

The optional Novas-2008??-demo.tar.gz package must be installed (where 2008 corresponds to the year and ?? corresponds to the month).

Installation and Setup

You must also complete the following actions in order to set up the Verdi environment and the files required for this tutorial:

1. Add the Verdi application (binary) to the search path and specify the search path to the license file:

Refer to the *Setting Up the Environment and Running the Software* section in the *Installation and System Administration Guide* for details.

2. Create a working directory:

```
% mkdir <working_dir>
```

3. All of the tutorial data resides in the `$VERDI_HOME/demo` directory. Make a copy of these demo files in your working directory:

```
% cp -r $VERDI_HOME/demo <working_dir>
```

The tutorial uses the case in the `$VERDI_HOME/demo/nECO` directory. Make sure the `demo/nECO` directory exists under your working directory.

Non-Freeze Silicon ECO

This tutorial illustrates the *nECO* operations in non-freeze silicon mode. In this mode, you have all the freedom to modify your design. This tutorial demonstrates how to insert a cell in original design, and edit connections.

Import a Verilog Gate Design

1. Change the directory to `<working_dir>/nECO/tutorial/gate`. Then set the environment variables for symbol libraries and execute the Verdi platform to import the sample CPU design:

```
> setenv NOVAS_LIBPATHSNOVAS_LIBPATHS $VERDI_HOME/share/symlib/xx
```

where `xx` stands for the bit of your machine. It is either 32 or 64.

```
> setenv NOVAS_LIBSNOVAS_LIBS default_u
```

2. Invoke the Verdi platform:

```
> verdi -f run.f &
```

The Verdi platform opens and the gate level design is imported.

Isolate the Logic for Modification

1. Change the scope to *system.i_cpu.i_ALUB*:

- In the **<Inst._Tree>** tab of the design browser frame, click the plus sign of *i_cpu* in the hierarchy browser pane.
- Double-click *i_ALUB* in the design browser frame.

The scope changes to *i_ALUB*. Module *ALUB* is displayed in the source code frame.

2. Find instance *system.i_cpu.i_ALUB.U250*:

- Type *U250* in the **Find String** text field on the toolbar.
- Press the carriage return to search for the first occurrence.

You see the instance *system.i_cpu.i_ALUB.U250* is highlighted.

3. Extract the partial connectivity schematic for *U250* by choosing the **Tools -> New Schematic -> Connectivity** command in the main window.

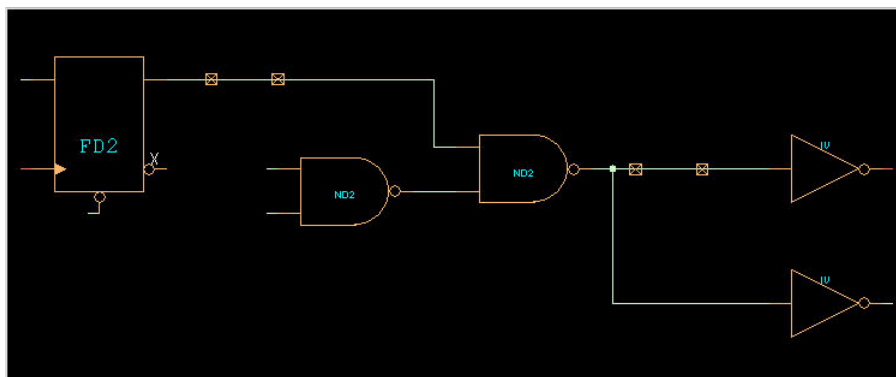
A new schematic frame opens. The driver and load logic of *U250* are extracted and displayed in it.

4. Invoke the *nSchema* frame for ECO modifications:

- In the *nSchema* frame, invoke the **Tools -> New Schematic -> ECO Window for All Instances** command.
- Click **Yes** on the *Question* form.

A new schematic frame appears. This is the *nECO* frame in which you can make the modifications.

Figure 2 *nECO* Frame of Partial Schematic



Check Hierarchies and Insert Buffers

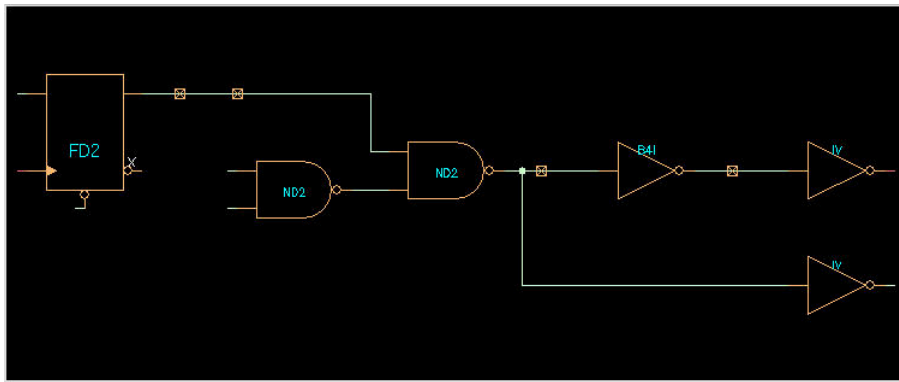
1. Check the design hierarchies and process the hierarchy connector.
 - a. In the *nECO* frame, turn on (check) the **Edit -> Enable Hierarchy Connector Editing** toggle command (if it is not on).
 - b. Select the hierarchy connector *system.i_cpu.i_ALUB.S1* (small box with an X to the right of *U250*).

Click-right to open the context menu to process the connector.

2. Add a buffer in the outer scope.
 - a. In the context menu, select the **Add Buffer** command.
 - b. In the initial *Add Buffer* form, select the **Outer Pin** option and click **OK**.
 - c. In the next *Add Buffer* form, select buffer *B4I*, and then click **OK**.
 - d. Click the **Keep Placement** icon  on the toolbar to rearrange the schematic.

The buffer is added to the outer hierarchy to help eliminate the delay.

Figure 3 Buffer B4I Added on Outer Hierarchy



Insert an Inverter between Instances

Insert an inverter between *system.i_cpu.i_ALUB.U295* and *system.i_cpu.i_PCU.U189*.

1. In the *nECO* frame, turn on (check) the **View -> Instance Name** toggle command to see the instance names.
2. Right-click anywhere in the *nECO* frame to open the context menu and choose the **Add Instance** command.

The *Add Instance* form opens. You can specify the instance name, cell type, and scope name for the added instance.

3. In the *Add Instance* form, specify the instance name and destination scope:

- Instance Name: a1
- Destination Scope Name: *system.i_cpu.i_PCU*
- Cell Name: IV

The form is displayed as follows:

Figure 4 *Add Instance Form*

Instance Name:

Destination Scope

Name:

Cell List

Find:

Name:

4. Click **OK** and an inverter is added in the *nECO* frame.

5. Turn on the **View -> Port Name** toggle command.

6. Turn on the **View -> Local Net Name** toggle command.

7. Delete the net connected to *system.i_cpu.i_ALUB.U295.Z*:

a. Select net *system.i_cpu.i_ALUB.n737*.

b. Click the **Delete Connection** icon  on the toolbar (or right-click and select the **Delete Connection** command from the context menu).

The net *n737* is deleted and Xs are marked on the pins that are connected to net *n737*. The mark indicates the pin has no connection.

8. Tie up *system.i_cpu.i_ALUB.U250.B*:

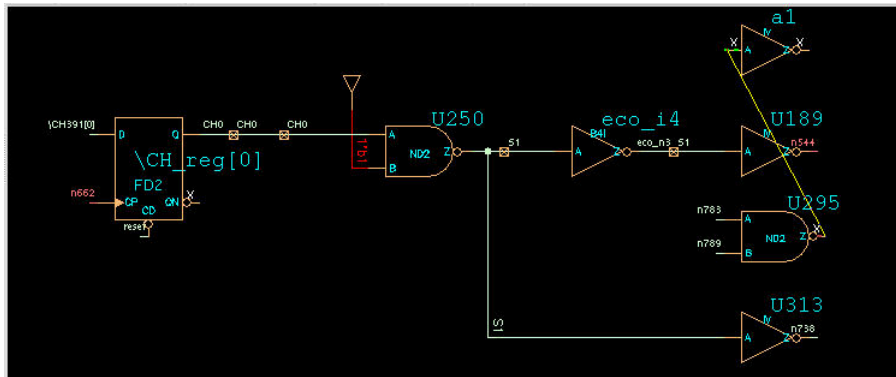
- Select pin *system.i_cpu.i_ALUB.U250.B*.
- Right-click to open the context menu and select the **Tie Up** command.

Now Pin B is tied up.

9. Connect *system.i_cpu.i_ALUB.U295.Z* and *system.i_cpu.i_PCU.a1.A*:

- Select pin *system.i_cpu.i_ALUB.U295.Z*.
- Right-click to open the context menu and select the **Make Connection** command (or click the **Make Connection** icon  on the toolbar).
- Move the cursor to *system.i_cpu.i_PCU.a1.A* and click-left.

Figure 5 Make Connection of U295.Z and a1.A



After connecting the two pins, a *New Ports/Nets* form opens to specify the net name. You can modify net name in this form. Since the two instances are in different scopes, one port is created on each module, also.

10. Click **OK** on the *New Ports/Nets* form.

Figure 6 New Ports/Nets Form

Module Name	Direction	Port Name
ALUB	Output	eco_n4
PCU	Input	eco_n4

Scope Name	Net Name
system.i_cpu	eco_n4

☐ Use Unique Name: eco_n4

OK Cancel

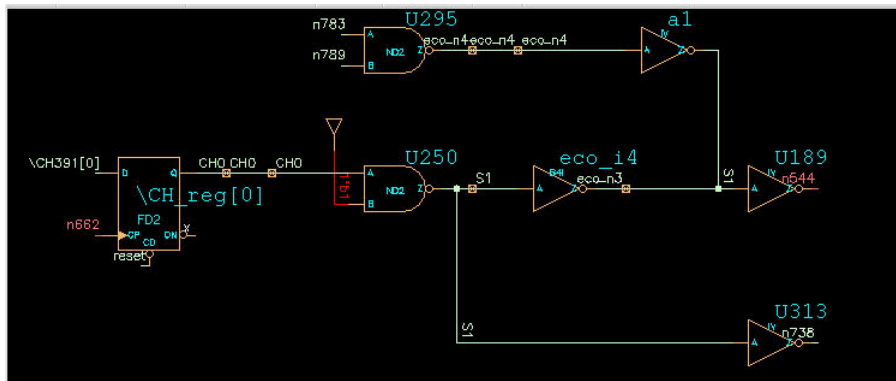
11. Connect *system.i_cpu.i_PCU.a1.Z* and *system.i_cpu.i_PCU.U189.A*:
 - a. Select pin *system.i_cpu.i_PCU.a1.Z*.
 - b. Right-click to open the context menu and select the **Make Connection** command (or click the **Make Connection** icon  on the toolbar).
 - c. Move cursor to *system.i_cpu.i_PCU.U189.A* and click-left.

Since *system.i_cpu.i_PCU.U189.A* is already connected to net *S1*, pin *a1.Z* is connected to *S1* also.

Change Placement and Routing

To optimize instance locations during editing, invoke the **Schematic -> Options -> Keep Placement** command or click the **Keep Placement** icon  on the toolbar. After this step, you see that the schematic is re-placed and re-routed by the *nECO* module with optimization. The default is *on*.

Figure 7 Re-place and RE-rout the Partial Schematic



Commit Change

1. Commit the modification, by invoking the **File -> Commit Change** command in the *nECO* frame.

The *Commit Changes Report* form opens. If you forget to commit changes, you see a message window to remind you when you exit the *nECO* frame. You can view the modifications you made for **Affected Modules&Instances**, **Instance Pin**, **Affected Net**, **Module**, **Instance**, and **Net/Port** during this session by selecting the related tab.

The form is displayed as follows:

Figure 8 Commit Changes Report Form

Affected Modules&Instances	Instance pin*	Affected net*	Module	Instance	Net / Port
ALUB	system.i_cpu.i_ALUB				
CPU	system.i_cpu				
PCU	system.i_cpu.i_PCU				

☒ In ECO Window
☐ Not in ECO Window

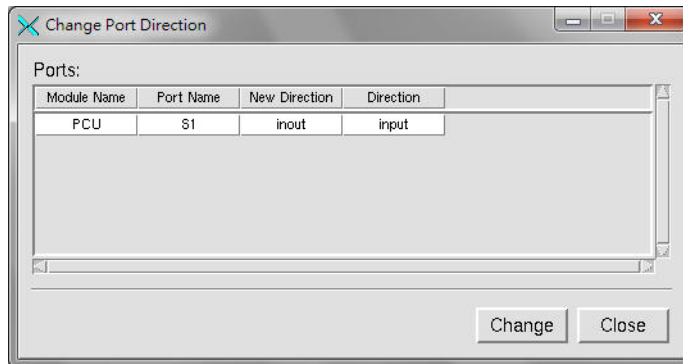
Comment:
Novas ECO updated by allen_shieh Dec 1, 2011 17:01

Commit Cancel

2. In the *Commit Changes Report* form, click **Commit** to commit the changes.

A *Change Port Direction* form opens asking you to change the port direction for port S1.

Figure 9 Change Port Direction



3. In the *Change Port Direction* form, click **Change** to confirm.
4. Close the *nECO* frame by choosing the **File -> Close Window** command.

Review Source Code

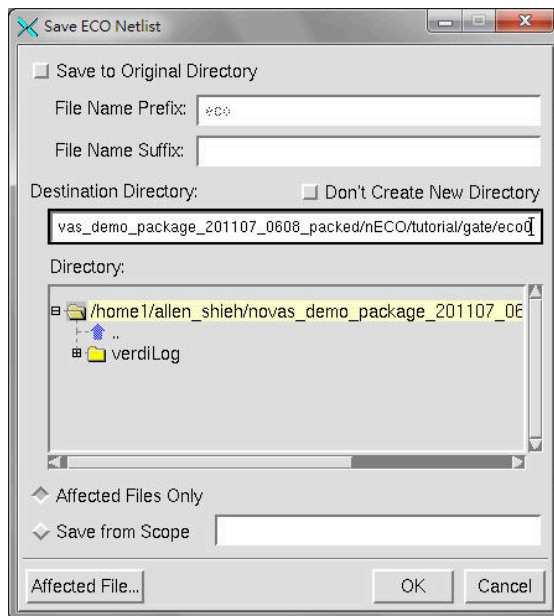
To review the source code in *nTrace*, perform the following:

1. Change the scope to *system.i_cpu.i_ALUB* by double-clicking *system.i_cpu.i_ALUB* in the **<Inst.Tree>** tab of the design browser frame.
You see a new port *eco_n_2* and a new buffer *B4I* added.
Now, you want go back to *i_PCU* to see what have changed there.
2. Double-click *i_PCU* in the design browser frame.
You see a new port *eco_n_2* and a new instance *a1* added on the source code frame.

Save ECO Results to a File

1. To save the modification to an HDL file, do the following:
 - a. In the main window, choose the **File -> ECO -> Save ECO Netlist** command.

Figure 10 Save ECO Netlist Form



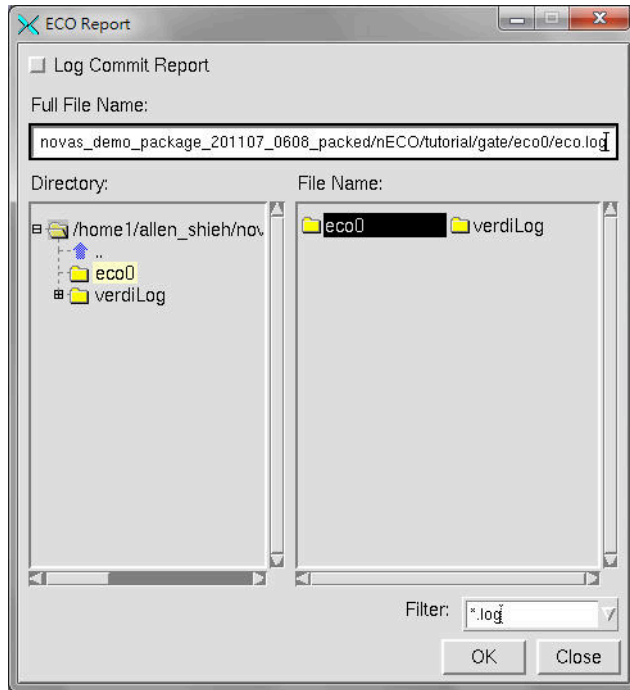
- b. In the *Save ECO Netlist* form, specify the destination directory. The default is `eco0` the first time an ECO is created in the directory. Each subsequent ECO increases the number, for example, `eco1`, `eco2`, and so on.
 - c. Select the **Affected Files Only** option.
 - d. Click **OK**.

The *nECO* module saves the *CPU.vg* file under the specified directory. If there is a *CPU.vg* file in that directory already, then *nECO* module asks for confirmation to overwrite the file. If you forget to do so, you see a message window to remind you.

2. Output the ECO report:
 - a. In the main window, invoke the **File -> ECO -> ECO Report** command.
 - b. Append `eco0/eco.log` to the **Full File Name** text field.
 - c. Click **OK**.

The *nECO* module keeps the history of all committed changes:

Figure 11 ECO Report Form



3. Review the ECO report:

```
> cd eco0  
> vi eco.log
```

You can view the record of all your modifications.

4. Exit the Verdi platform by selecting the **File -> Exitcommand**.

Freeze Silicon ECO

This tutorial illustrates *nECO* operations in freeze silicon mode. In this mode, you can only use spare cells for new instances and a deleted instance is set as a spare cell for future use.

Import a Verilog Design

1. Change the directory to `<working_dir>/tutorial/gate`. Then set the environment variables for symbol libraries and execute the Verdi platform to import the sample CPU design:

```
> setenv NOVAS_LIBPATHS $VERDI_HOME/share/symlib/xx
```

where xx stands for the bit of your machine. It is either 32 or 64

```
> setenv NOVAS_LIBS default_u  
> verdi -f run.f &
```

2. Invoke the Verdi platform:

```
> verdi -f run.f &
```

The Verdi platform opens and the gate level design is imported.

Manage Spare Cell by Instance Name

Turn on Freeze Silicon Mode

Manage Spare Cell by Instance Name

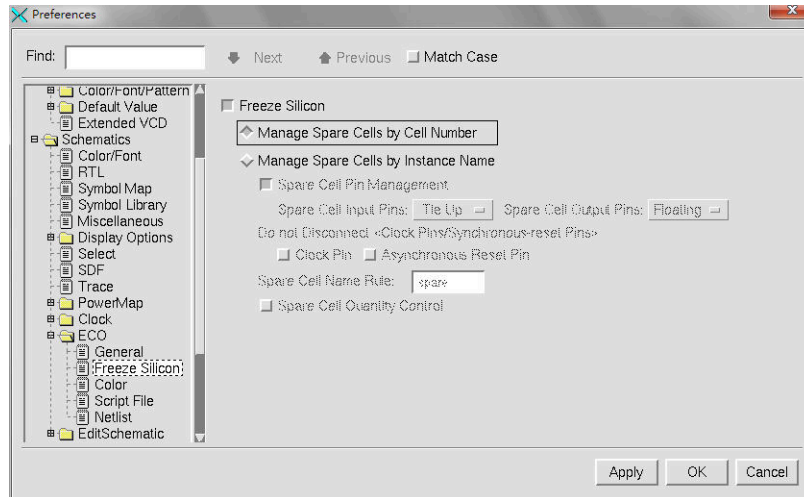
Turn on Freeze Silicon Mode

To turn on Freeze Silicon mode perform the following steps:

1. In the main window, invoke the **Tools -> Preferences** command to open the *Preferences* form.
2. In the *Preferences* form, select the **Schematics** page -> **ECO** page -> **Freeze Silicon** page.
3. Turn on the **Freeze Silicon** option.
4. Turn on the **Manage Spare Cell by Instance Name** option.
5. Turn on the **Spare Cell Pin Management** option.
6. Choose **Tie Up** for **Spare Cell Input Pins**.
7. Choose **Floating** for **Spare Cell Output Pins**.
8. Type *spare* in the **Spare Cell Name Rule** text field.
9. Click **OK**.

The *Preferences* form is displayed as follows:

Figure 12 ECO Preference Settings



The above settings specify this modification is in freeze silicon mode. If there are any instances deleted in this session, the instances are renamed as *spare**. The input pins are tied-up automatically.

Isolate the Logic for Modification

1. Change the scope to *system.i_cpu.i_ALUB*.
 - a. In the **<Inst.Tree>** tab of the design browser frame, click the plus sign of *i_cpu*.
 - b. Double-click *i_ALUB* in the design browser frame.

The scope changed to *system.i_cpu.i_ALUB* and module *ALUB* is displayed in the source code frame.

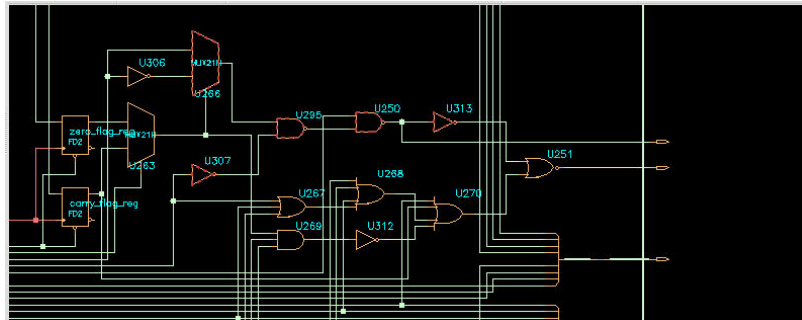
2. Invoke the *nSchema* frame for scope *ALUB* by clicking the **New Schematic** icon on the toolbar.

A new schematic frame opens displaying the schematic for module *ALUB*.

3. Select instances for the ECO:
 - a. Zoom in on the bottom-right corner.
 - b. In the *nSchema* frame, turn on the **View -> Instance Name** toggle command.
 - c. Select *U266*, *U307*, *U295*, *U250*, and *U313*.

You can use <Shift> + click-left to select multiple objects in *nSchema*.

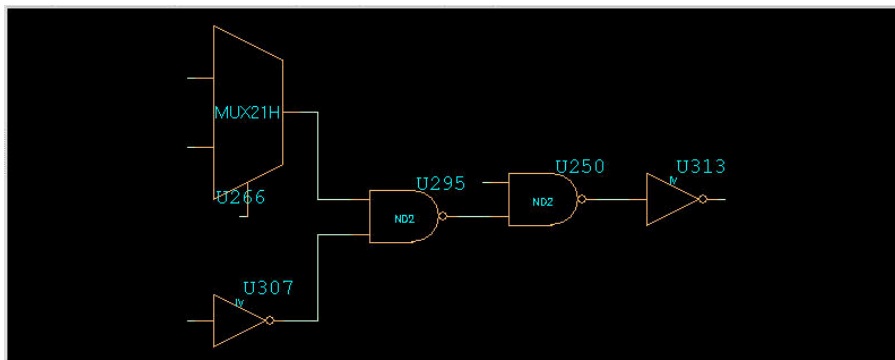
Figure 13 Zoom in to Bottom-Right Corner of *i_ALUB*



4. Open the *nECO* frame for the selected instances by choosing the **Tools -> New Schematic -> ECO Window for Selected Instance(s)** command, and click **Yes** on the *Question* form.

A new schematic frame opens with the selected objects. This is the *nECO* frame in which you can make the modifications.

Figure 14 *nECO* frame for Partial Schematic



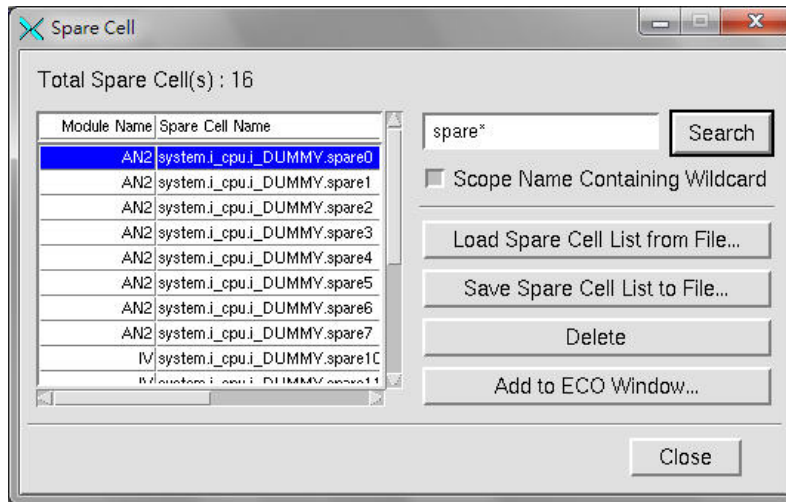
List Spare Cells

1. Right-click in the *nECO* frame to open the context menu and select the **Spare Cell** command. The *Spare Cell* form opens.
2. In the *Spare Cell* form, add wildcard "*" to the default string in the text field to the left of the **Search** button.
3. Turn on the **Scope Name Containing Wildcard** option.
4. Click **Search**.

All instances named *spare** are added as spare cells.

There are two ways to specify spare cells in your design. You can specify name pattern for listing spare cells or prepare a cell list file and load the file to specify spare cells.

Figure 15 Spare Cell form - Before Replace Instance



5. Click **Close** to close the *Spare Cell* form.

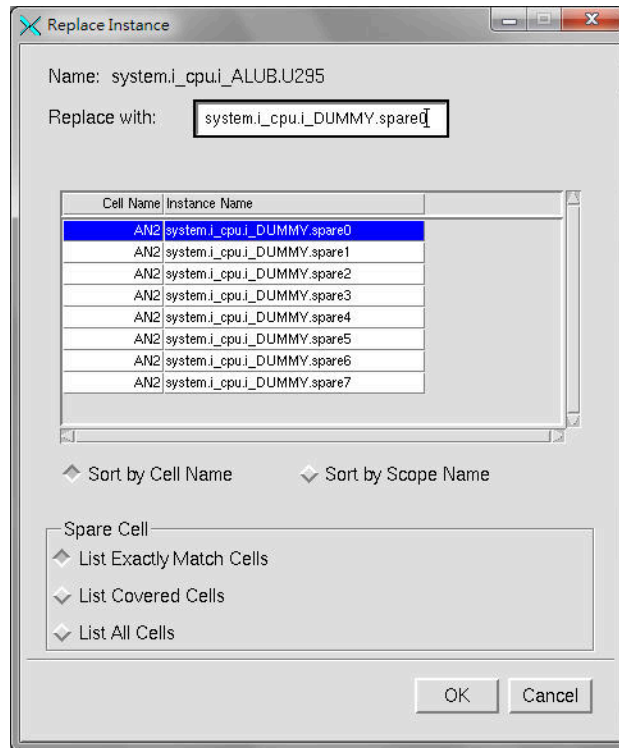
Replace Current Cell Type with Another Cell Type

Replace the *i_ALUB.U295* cell type from ND2 to AN2.

1. Replace the instance:
 - a. In the *nECO* frame, select instance *i_ALUB.U295*.
 - b. Right-click to open the context menu and select the **Replace Instance** command.

The *Replace Instance* form is displayed as follows. You have to choose a cell from the list since you are in freeze silicon mode.

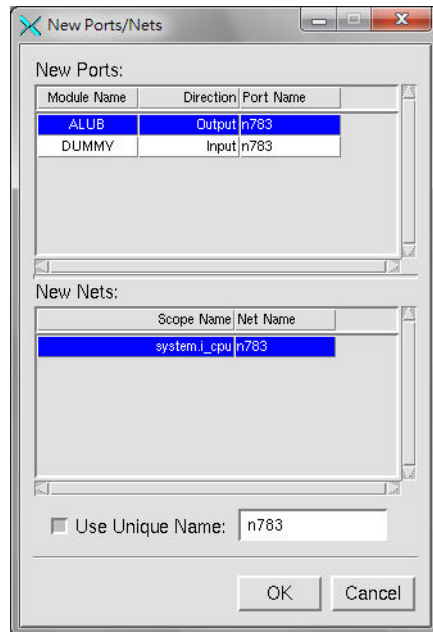
Figure 16 *Replace Instance Form*



2. Specify the instance to replace:
 - a. Choose *system.i_cpu.i_DUMMY.spare0*.
 - b. Click **OK**.

The *New Ports/Nets* form opens indicating new ports/nets *n783*, *n789*, and *n737* are added.

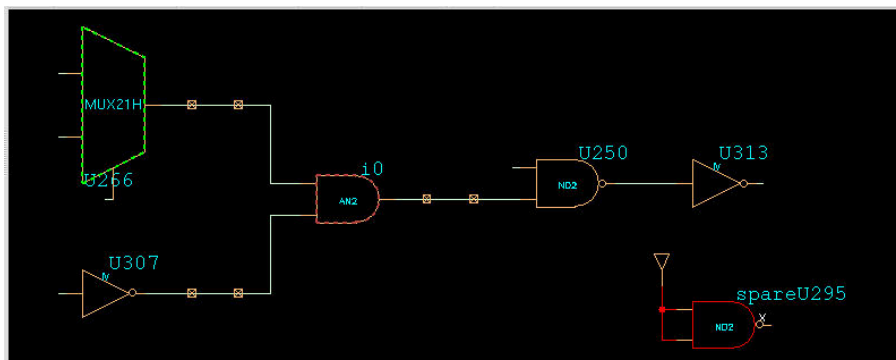
Figure 17 Figure: New Ports/Nets Form



3. Click **OK** on the *New Ports/Nets* forms.

In the *nECO* frame, you see the original instance *system.i_cpu.i_ALUB.U295* is replaced by *AN2* and the original *ND2* cell is renamed to *system.i_cpu.i_ALUB.spareU295*. The input pins of the *ND2* are tied up and output pin is floating as specified in the *Preferences* form.

Figure 18 Replace Instance

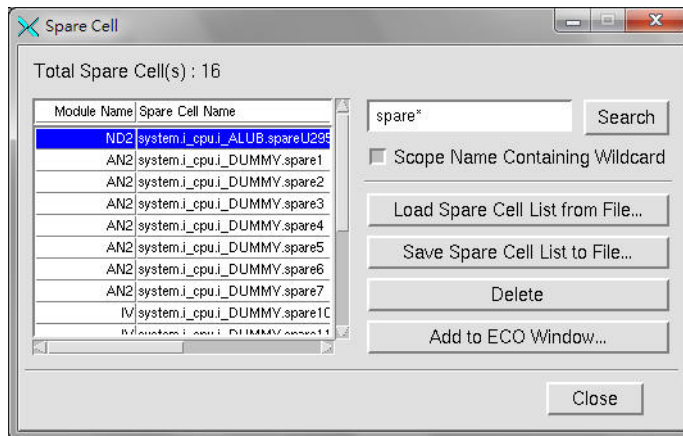


Check and Save Spare Cells List

1. In the *nECO* frame, right-click to open the context menu and select the **Spare Cell** command.

The *Spare Cell* form opens. You can see that *spare0* is removed from the list and *spareU295* is added into the spare cell list. That is, you can use it in a future modification.

Figure 19 Spare Cell Form - After Replace Instance



2. Save the spare cell list to file:
 - a. In the *Spare Cell* form, click **Save Spare Cell List to File**.
 - b. In the *Spare Cell List* form, specify the file name as *spare_list*.
 - c. Click **OK**.
 - d. In the *Spare Cell* form, click **Close**.

You can load *spare_list* file next time to manage spare cell list.

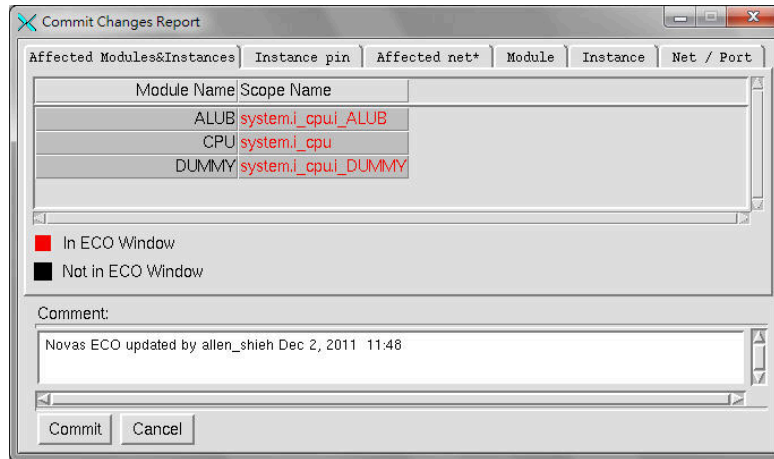
Commit Change

1. Commit the modification by choosing the **File -> Commit Change** command in the *nECO* frame.

The *Commit Changes Report* form opens. If you forget to commit changes, you see a message window to remind you when you exit the *nECO* frame. You can view the modifications you made for **Affected Modules & Instances**, **Instance Pin**, **Affected Net**, **Module**, **Instance**, and **Net/Port** during this session by selecting the related tab.

The form is displayed as follows:

Figure 20 Commit Changes Report



2. In the *Commit Changes Report* form, click **Commit**.
3. Close the *nECO* frame by choosing the **File -> Close Window** command.

Review Source Code

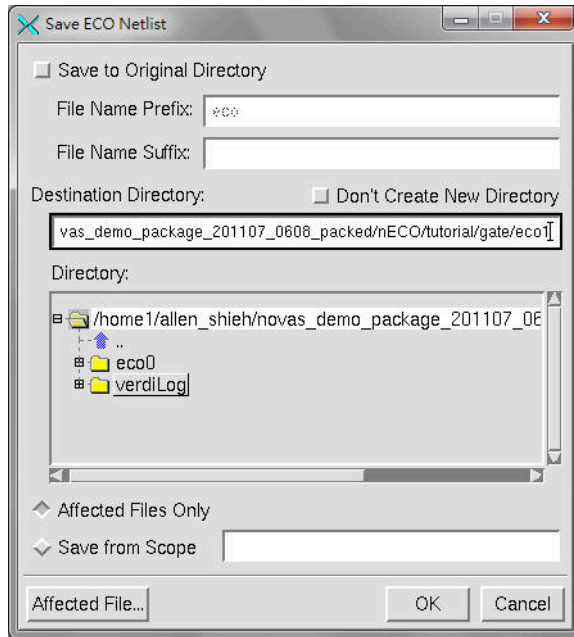
To review the source code, change the scope to *system.i_cpu.i_ALUB* by double-clicking on *system.i_cpu.i_ALUB* in the design browser frame. You see the original instance *U295* is replaced by *system.i_cpu.i_dummy.i0* with cell type *AN2* and *ND2* is now *spareU295*.

Save ECO Results to a File

1. Save modifications to a file:
 - a. In the main window, choose **File -> ECO -> Save ECO Netlist**.
 - b. In the *Save ECO Netlist* form, specify the directory name as *eco1*.
 - c. Select the **Affected Files Only** option.
 - d. Click **OK**.

The *nECO* module saves *CPU.vg* file under the specified directory, *eco1*. If there is *CPU.vg* in that directory, the *nECO* module asks for confirmation to overwrite the file. If you forget to do so, you see a message window to remind you.

Figure 21 Save ECO Netlist Form



2. Exit the Verdi platform by choosing the **File -> Exit** command.

Manage Spare Cell by Cell Number

Turn on Freeze Silicon Mode

To turn on Freeze Silicon mode, perform the following steps:

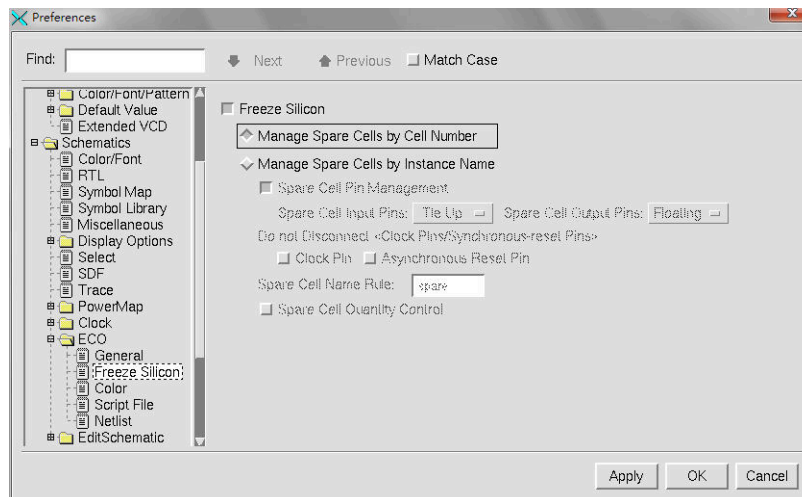
1. In the current directory, invoke the Verdi platform as follows:

```
verdi -f run.f
```
2. In the main window, invoke the **Tools -> Preferences** command to open the *Preferences* form.
3. In the *Preferences* form, select the **Schematics** page -> **ECO** page -> **Freeze Silicon** page.
4. Turn on the **Freeze Silicon** option.
5. Turn on the **Manage Spare Cell by Cell Number** option.

6. Click **OK**.

The above settings specify this modification are in freeze silicon mode.

Figure 22 ECO Preference Settings



Isolate the Logic for Modification

1. Change scope to *system.i_cpu.i_ALUB*.

- In the **<Inst.Tree>** tab of the design browser frame, click the plus sign in front of *i_cpu*.
- Double-click *i_ALUB* on the design browser frame.

The scope changed to *system.i_cpu.i_ALUB* and module *ALUB* is displayed in the source code frame.

2. Invoke the *nSchema* frame on scope *ALUB* by clicking the **New Schematic** icon on the toolbar.

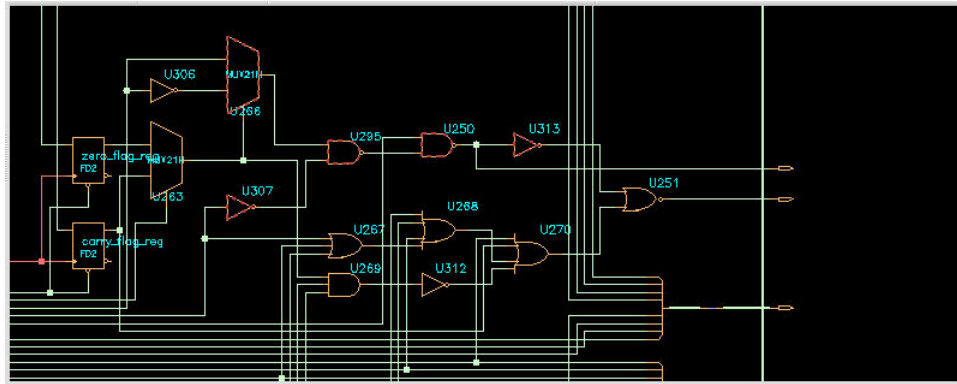
A new schematic window opens displaying the schematic for module *ALUB*.

3. Select instances for the *nECO* frame:

- Zoom in on the bottom-right corner.
- Turn on the **View -> Instance Name** toggle command.
- Select *U266*, *U307*, *U295*, *U250*, and *U313*.

You can use <Shift> + click-left to select multiple objects in the *nSchema* frame.

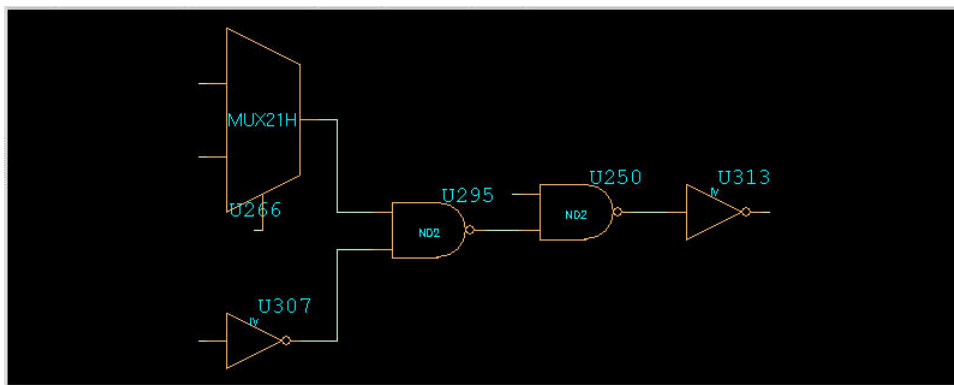
Figure 23 Zoom in to Bottom-Right Corner of *i_ALUB*



4. Open the *nECO* frame for the selected instances by choosing the **Tools -> New Schematic -> ECO Window for Selected Instance(s)** command, and click **OK** on the *Question* form.

A new schematic window opens with the selected objects. This is the *nECO* frame in which you can make the modifications.

Figure 24 *nECO* Frame for Partial Schematic



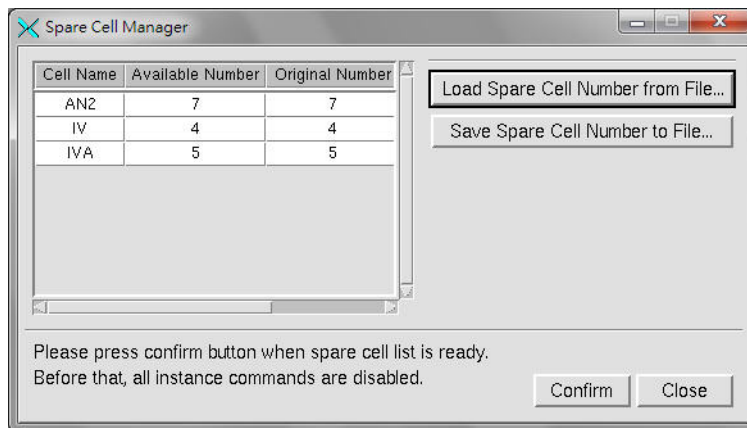
Load Spare Cell Number from File

1. In the current working directory, use your favorite editor to create a spare cell file list called, *cell_list*, with the following content:

```
IV 4
AN2 7
IVA 5
```

2. In the *nECO* frame, right-click to open the context menu and select the **Spare Cell** command. The *Spare Cell Manager* form opens.
3. In the *Spare Cell Manager* form, click **Load Spare Cell Number from File**.
4. In the *Load Spare Cell Number From File* form, specify the *cell_list* file name and click **OK**.
5. In the *Spare Cell Manager* form, click **Confirm**.

Figure 25 Spare Cell Manager Form



Replace Current Cell Type with New Cell Type

Replace the *i_ALUB.U307* instance cell type from IV to IVA.

1. Replace the instance:
 - a. In the *nECO* frame, select instance *i_ALUB.U307*.
 - b. Right-click to open the context menu and select the **Replace Instance** command.The *Replace Instance* form opens. You can choose a cell from the list since you are in freeze silicon mode.

2. Specify the instance to replace:
 - a. In the *Replace Instance* form, enter *IVA* in the **Replace with** text field.
 - b. Click **OK**.

Figure 26 *Replace Instance Form*

Replace Instance

Name: system.i_cpui_ALUB.U307

Replace with: IVA

Cell Name	Available Number
IVA	5

☐ Sort by Cell Name ☐ Sort by Available Number

Spare Cell

☐ List Exactly Match Cells

☐ List Covered Cells

☐ List All Cells

OK Cancel

In the *Spare Cell Manager* form, you see the number of *IV* cell types decreases to 4 and the number of *IVA* cell types increased to 5.

Figure 27 *Spare Cell Manager after Replace Instance*

Spare Cell Manager

Cell Name	Available Number	Original Number
AN2	7	7
IV	5	4
IVA	4	5

Load Spare Cell Number from File...

Save Spare Cell Number to File...

Please press confirm button when spare cell list is ready.
Before that, all instance commands are disabled.

Confirm Close

Save Spare Cells List

1. Save spare cell list to file:
 - a. In the *Spare Cell Manager* form, click **Save Spare Cell Number to File**.
 - b. Specify the file name as *cell_list1*.
 - c. Click **OK**.

You can load this file next time to manage the spare cell number.

Commit Change

1. Commit the modification by choosing the **File -> Commit Change** command in the *nECO* frame.

The *Commit Changes Report* form opens. If you forget to commit changes, a message window is displayed to remind you when you exit the *nECO* frame. You can view the modifications for **Affected Modules & Instances**, **Instance Pin**, **Affected Net**, **Module**, **Instance**, and **Net/Port** during this session by selecting the related tab.

The form is displayed as follows:

Figure 28 *Commit Changes Report Form*

Module Name	Scope Name
ALUB	system.i_cpui_ALUB

☒ In ECO Window
☐ Not in ECO Window

Comment:
Novas ECO updated by allen_shieh Dec 2, 2011 14:26

Commit Cancel

2. In the *Commit Changes Report* form, click **Commit**.
3. Close the *nECO* frame by choosing the **File -> Close Window** command.

Review Source Code

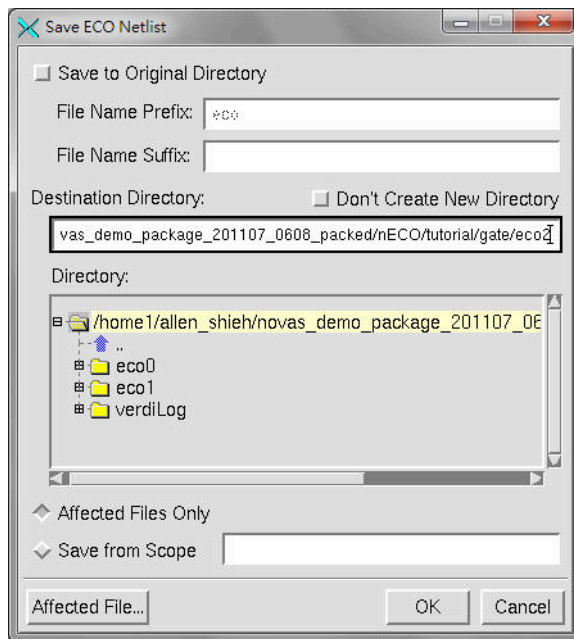
To review the source code, change the scope to *system.i_cpu.i_ALUB* by double-clicking on *system.i_cpu.i_ALUB* in the design browser frame. In the source code frame, you see the original instance *U307 (IV)* is replaced with cell type *IVA*.

Save ECO Results to a File

1. Save modifications to a file:
 - a. In the main window, choose the **File -> ECO -> Save ECO Netlist** command.
 - b. In the *Save ECO Netlist* form, specify the directory name as *eco2*.
 - c. Turn *on* the **Affected Files Only** option.
 - d. Click **OK**.

The *nECO* module saves the *CPU.vg* file under the specified directory, *eco2*. If there is a *CPU.vg* file in that directory, the *nECO* module asks for confirmation to overwrite the file. If you forget to do so, you see a message window to remind you.

Figure 29 Save ECO Netlist Form



2. Exit the Verdi platform by choosing the **File -> Exit** command.

Clone a Module

This tutorial shows useful *nECO* utilities to help you modify your design, clone a module, and move an instance to another scope - all from the *nSchema* window. This is a post place and route case. Due to timing issues, you have to size an inverter inside an instance in freeze silicon mode.

Import Your Design

You must perform the *Freeze Silicon ECO* tutorial before completing this tutorial.

1. Use the demo case under `<working_dir>/tutorial/gate`:

Link *eco1/CPU.vg*:

```
> mv CPU.vg CPU.vg.v0  
> ln -s eco1/CPU.vg CPU.vg
```

2. Import the design:

```
> verdi -f run.f &
```

The Verdi platform opens and the gate-level design is imported.

Check the Preferences Setup

You turned on **Freeze Silicon Mode** in a previous tutorial. Verify the setup as follows:

1. In the main window, invoke the **Tools -> Preferences** command to open the *Preferences* form.
2. In the *Preferences* form, select the **Schematics** page -> **ECO** page -> **Freeze Silicon** page.
3. Turn on the **Freeze Silicon** option.
4. Turn on the **Manage Spare Cell by Instance Name** option.
5. Turn on the **Spare Cell Pins Management** option.
6. Choose **Tie Up** for **Spare Cell Input Pins**.
7. Choose **Floating** for **Spare Cell Output Pins**.
8. Type *spare* in the **Spare Cell Name Rule** text field.
9. Click **OK**.

Isolate the Logic for Modification

1. Change scope to *system.i_cpu.i_ALUB.i_alu.sub_25_1*:
 - a. In the **<Inst.Tree>** tab of the design browser frame, click the plus signs before *i_cpu*, *i_ALUB*, and *i_alu*.
 - b. Double-click *sub_25_1* on the design browser frame.

The scope changes to *system.i_cpu.i_ALUB.i_alu.sub_25_1* and module *alu_DW01_sub_9* is displayed in the source code frame.

2. Invoke the *nSchema* window for the *alu_DW01_sub_9* scope by clicking the **New Schematic** icon on toolbar.

A new *nSchema* frame opens to display the schematic of module *alu_DW01_sub_9*.

Clone Module

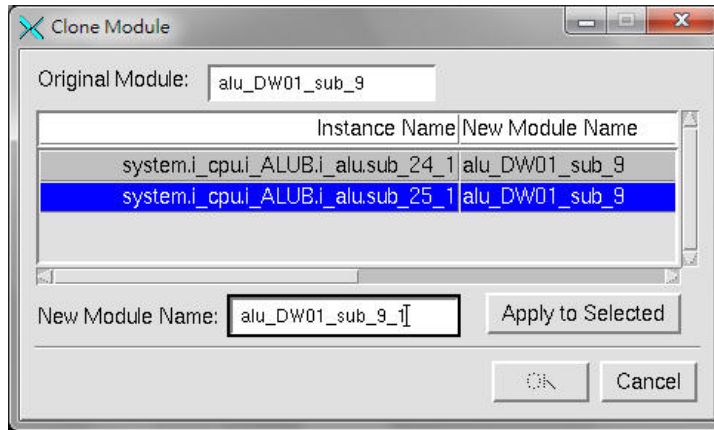
Clone the *alu_DW01_sub_9* module.

1. Before you do the modification, confirm this module is not instantiated multiple times.
 - a. In the *nSchema* window, choose the **Tools -> ECO Utility -> Clone Module** command.
 - b. In the *Clone Module* form, enter *alu_DW01_sub_9* with carriage return in **Original Module** text field.

There are two instances listed in *Clone Module* form. That is, the two instances refer to the same module *alu_DW01_sub_9*. Since your change is only needed for instance *sub_25_1*, you have to clone this module.
2. Specify the new module name.
 - a. In the *Clone Module* form, select instance *system.i_cpu.i_ALUB.i_alu.sub_25_1* in the list.
 - b. Enter *alu_DW01_sub_9_1* in **New Module Name** text field.

The form is displayed as follows:

Figure 30 Clone Module Form



c. Click **Apply to Selected**.

d. Click **OK**.

The *Commit* window opens.

e. In the *Commit* window, click **OK** to accept the comment for **Clone Module**.

Now, the module for instance *sub_25_1* is *add_DW01_sub_9_1* and you can do the modification.

Load Spare Cell List

You are in freeze silicon mode, so before adding or replacing instances, you have to load a spare cell list.

1. In the *nSchema* frame, choose the **Tools -> ECO Utility -> Spare Cell** command.
2. In the *Spare Cell* form, click **Load Spare Cell List From File**.
3. In the *Load Spare Cell From File* form, select file *spare_list* and click **OK**.
4. In the *Spare Cell* from, click **Close**.

You have already used a spare cell and saved a new spare list to the *spare_list* file in the previous tutorial. Now, you can just load the file, *spare_list*.

Size the Inverter

1. Select the inverter and invoke the *nECO* frame.
 - a. In the *nSchema* window, turn on (check) the **View -> Instance Name** toggle command.
 - b. Select *U3* in the lower right corner.
 - c. Choose the **Tools -> New Schematic -> ECO Window for Selected Instance(s)** command, and click **Yes** on the *Question* form.

A new schematic frame opens with instance *U3*. This is the *nECO* frame in which you can make the modifications.

2. Replace the instance:
 - a. Select *U3* and right-click to open the context menu and select the **Replace Instance** command.

The *nECO* frame lists all available instances for replacement. You want to resize instance *U3*, which is an inverter so choose *IVA* to replace.
 - b. In the *Replace Instance* form, select *system.i_cpu.i_cpu.i_DUMMY.spare14 (IVA)*.
 - c. Click **OK**.

The *New Ports/Nets* form opens several times indicating new ports/nets were added.

- d. In each *New Ports/Nets* form, click **OK**.

In the *nECO* frame, you can see the original instance *U3* is renamed as *i14* and changed to cell type *IVA*. The original cell *IV*, is treated as a spare cell and renamed as *spareU3*.

Commit Change

1. Commit the modification by choosing the **File -> Commit Change** command in the *nECO* frame.

The *Commit Changes Report* form opens. If you forget to commit changes, a message window is displayed to remind you when you exit the *nECO* frame. You can view the modifications made for **Affected Modules&Instances, Instance Pin, Affected Net, Module, Instance**, and **Net/Port** during this session by selecting the related tab.

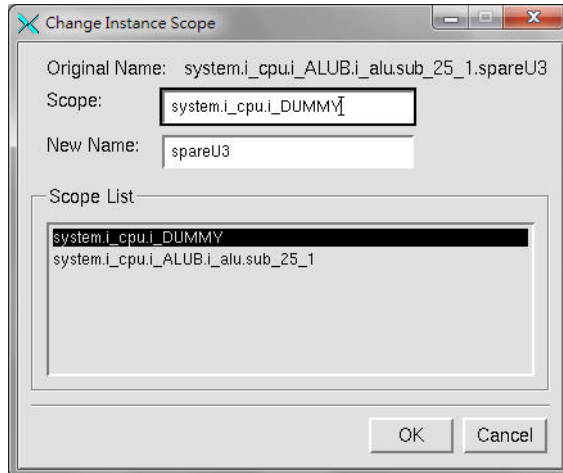
2. In the *Commit Changes Report* form, click **Commit**.

Move Spare Cell to Another Scope

In the *nSchema* window for *sub_25_1*, a new instance *spareU3* is added. This is the replaced instance. You can move it to scope *i_DUMMY* to make the design more clear.

1. In the *nECO* frame, select *spareU3*.
2. Right-click to open the context menu and select the **Change Instance Scope** command.
3. In the *Change Instance Scope* form, enter *system.i_cpu.i_DUMMY* in the **Scope** text field.

Figure 31 *Change Instance Scope Form*



4. Click **OK**.
5. In the *nECO* frame, choose the **File -> Commit Change** command to commit the change and complete the move.

You see instance *spareU3* is removed. The source code is updated at the same time.

Save HDL Code to a File

1. Save modifications to a file.
 - a. In the main window, choose the **File -> ECO -> Save ECO Netlist** command.
 - b. In the *Save ECO Netlist* form, specify the directory name as *eco3*.
 - c. Turn on the **Affected Files Only** option.
 - d. Click **OK**.

The *nECO* module saves the *CPU.vg* file under the specified directory, *eco3*. If there is a *CPU.vg* file in that directory, the *nECO* module asks for confirmation to overwrite the file. If you forget to do so, you see a message window to remind you.

2. Exit the Verdi platform by choosing the **File -> Exit** command.

Change Instance Scope

Sometimes, you may need to re-structure your design due to timing or place and route issues. When you move instances to another scope, you have to deal with many interface problems, such as maintaining connections, deleting ports, and adding ports. This tutorial shows you how the *nECO* frame can help you with this.

Import Your Design

1. Change the directory to `<working_dir>/tutorial/gate`. Then set the environment variables for symbol libraries and execute the Verdi platform to import the sample CPU design:

```
> setenv NOVAS_LIBPATHS $VERDI_HOME/share/symlib/xx
```

where `xx` stands for the bit of your machine. It is either 32 or 64

```
> setenv NOVAS_LIBS default_u
```

2. Invoke the Verdi platform:

```
> verdi -f run.f &
```

Then the Verdi platform opens and the gate level design is imported.

Note:

Turn *off* the **Freeze Silicon** option on **Tools -> Preferences -> Schematics** page -> **ECO** page -> **Freeze Silicon** page before beginning this tutorial.

Change Instance Scope

1. In the main window, double-click `i_cpu` in the **<Inst.Tree>** tab of the design browser frame, to open the `system.i_cpu` scope. The scope changes to `i_cpu` and module `CPU` is displayed in the source code frame.
2. Invoke the *nSchema* window for scope `CPU` by clicking the **New Schematic** icon on toolbar.

A new *nSchema* frame opens displaying the schematic for module `CPU`.

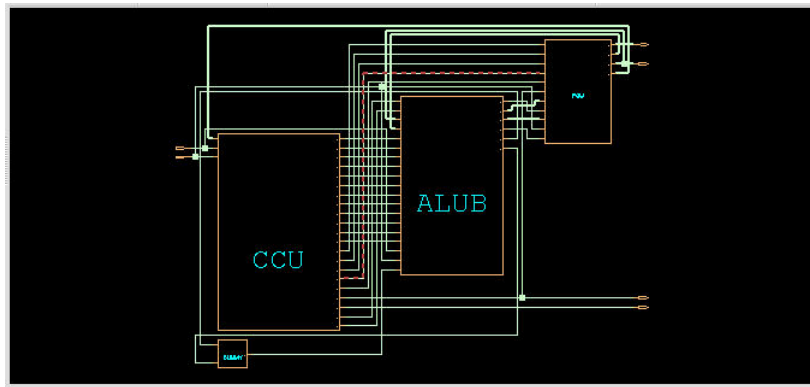
Assume there is a violated path with net `system.i_cpu.C6` in it. Try to eliminate this violated path by moving its instance scope to `i_CCU`.

3. Find the instance *system.i_cpu.C6*:

- In the *nSchema* window, turn on (check) the **Schematic -> Auto Fit Found Object(s)** toggle command.
- Select the **Schematic-> Find Signal/Instance/Instport** command.
- In the *Find Signals* form, enter *system.i_cpu.C6*.
- Click **Find** to select the path.

This path crosses two scopes, *i_CCU* and *i_PCU* which may be why there is a violated path.

Figure 32 Scope *system.i_cpu* with *system.i_cpu.C6* Highlighted



4. Trace the load of *system.i_cpu.C6* by double-clicking on the instance portC6 of *i_PCU*.

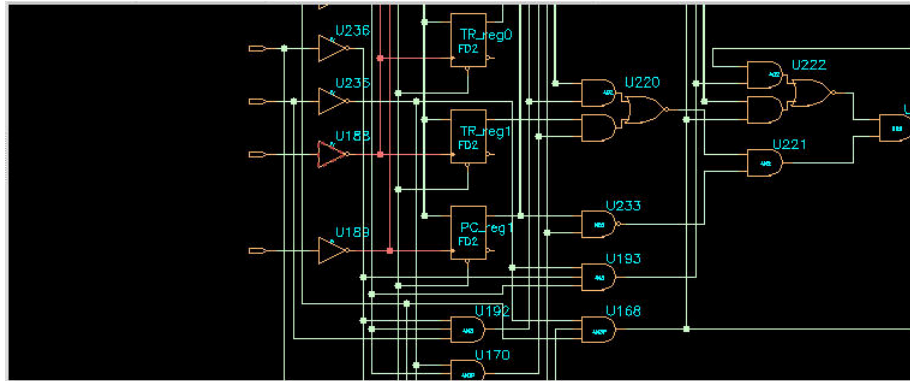
The schematic of module *i_PCU* is displayed in the *nSchema* frame now.

5. Select instance *U188*:

- Zoom in on the bottom-left corner.
- Turn on (check) the **View-> Instance Name** toggle command.

c. Select *U188*.

Figure 33 : Zoom in to Bottom-Left Corner of *i_PCU* and Get *U188*



You want to move this cell into scope *i_CCU*; however, let's check its connectivity first.

6. Extract the partial connectivity schematic for *U188* by selecting *U188* and choosing the **Tools -> New Schematic -> Connectivity** command.

A new *nSchema* window opens. As you can see, the output of *U188* is a clock, which is the input to 8 flip-flops. It is better that you also move these flip-flops into *i_CCU* to prevent clock skew.

7. Click the **Be Window** icon on the toolbar of the *View Trace Connectivity Result* schematic frame, to make it a standalone window.
8. In the *View Trace Connectivity Result* window, select *U188* and all of the loading flip-flops (choose the **Schematic -> Selection -> Select All Instances** command and then press the shift key and click-left on *C6_reg* to deselect it), then drag and drop them back to the previous *nSchema* frame.
9. In the *nSchema* frame, invoke the **Tools -> ECO Utility -> Change Instance Scope** command.
10. In the *Change Instance Scope* form, enter *system.i_cpu.i_CCU* in the **New Scope** text field (you can also drag and drop the new scope from the design browser frame) and click **OK**.
11. In the *Commit* form, click **OK**.

All of the selected cells are now moved into the expected scope, *i_CCU*.

Save HDL Code to a File

1. Save modifications to a file.
 - a. In the main window, choose the **File -> ECO -> Save ECO Netlist** command.
 - b. In the *Save ECO Netlist* form, specify the directory name as *eco4*.
 - c. Turn on the **Affected Files Only** option.
 - d. Click **OK**.

Then *nECO* module saves the *CPU.vg* file under the specified directory, *eco4*. If there is a *CPU.vg* file in that directory, the *nECO* module asks for confirmation to overwrite the file. If you forget to do so, a message window is displayed to remind you.

2. Exit the Verdi platform by choosing the **File -> Exit** command.

4

Appendix A: Novas ECO Script

To perform repetitive Engineering Change Order (ECO) changes in an easy and efficient way, the *nECO* module provides a set of Novas ECO script commands, which are programmed in the Tcl format. The ECO changes may be performed using both Novas ECO script commands and all other standard Tcl functions.

Commands and Key Components

This section describes two kinds of Novas ECO script commands and explains how to use these script commands for the ECOs.

Novas ECO script commands are categorized as two types and are presented with different prefixes. These two types are as follows:

- Commands with prefix `_eco`
- Commands with prefix `ECO`

The commands with prefix `_eco` must be included within the `ECOBeginSession` and `ECOEndSession` commands. Each ECO session begins with the `ECOBeginSession` command and ends with the `ECOEndSession` command. The `ECOEndSession` command not only means an ECO session is ended; it also implies a commit action. The *nECO* module does not commit the changes for the commands with the prefix `_eco` until it encounters the `ECOEndSession` command. Multiple ECO sessions can exist in a script file; however, nested ECO sessions are not allowed - an ECO session cannot be included in another ECO session.

The commands with prefix `ECO` must not be included within the above ECO session. These ECO commands perform the related ECO changes and imply an ECO commit action at the same time.

Prior to using the Novas ECO script commands for ECOs, a script file in the Tcl format, may be written with the provided Novas ECO commands. Since the Tcl format is adopted, all other standard Tcl commands may also be used in this script file.

After the script file is ready, an ECO may be performed by loading the file into the *nECO* window using the **File -> ECO -> Load ECO Script** command from *nTrace*. After loading the script file, use the Verdi standard functionality to verify the output of the ECO changes.

If the output result is correct, use the **File -> ECO -> Save ECO Netlist** command in *nTrace* to save the ECO changed HDL files onto disk.

Script File Format

The following table provides the Novas ECO script syntax and explains the usage of these scripts. The first line is the command syntax and the following lines in italics are examples of the command.

Note:

These are not the *nECO* Tcl commands. Refer to the *nECO* chapter in the *Verdi and Siloti Tcl Reference* for the complete list of Tcl commands.

Novas ECO Script Syntax	Description
<code>_ecoAddInst [-master mastername] -inst instname [-delim delim]</code>	Adds an instance to the design.
<i>_ecoAddInst -master "AD01D1" -inst "system.i_cpu.i_ALUB.eco_i_3"</i>	
<code>_ecoDeleteInst -inst instname [-delim delim]</code>	Removes an instance from the design.
<i>_ecoDeleteInst -inst "system.i_cpu1.i_ALUB.\X0[6] " "</i>	
<code>_ecoDeleteConnection -instport instName portName [-delim delim]</code>	Disconnects an instance port from a net. Note: A warning message is issued if the instance ports do not exist.
<i>_ecoDeleteConnection -instport "system.i_cpu1.i_ALUB.\X0[6] " "D"</i>	
<code>_ecoReplaceInst [-master mastername -sparecell sparecellname] -inst instname [-delim delim] [-map connection-info]</code>	Replaces an instance cell master with another cell master. Note: A warning message is issued if an instance does not exist.
<i>_ecoReplaceInst -master "YFD2" -inst "system.i_cpu1.i_ALUB.\X0[6] " _ecoReplaceInst -sparecell "system.i_cpu.i_PCU.U183" -inst "system.i_cpu.i_ALUB.U253" -map "A n742 system.i_cpu.i_ALUB.n742 Output system.i_cpu.path1 system.i_cpu.i_PCU.path1 Input" "B n743 system.i_cpu.i_ALUB.n743 Output system.i_cpu.path2 system.i_cpu.i_PCU.path2 Input" "Z \X0[6] system.i_cpu.i_PCU.path3 Output system.i_cpu.path3 system.i_cpu.i_ALUB.\X0[6] Input"</i>	

<pre>ecoMakeConnection [-map connection-info] [-instport instName portName [instName portName -signal netName -tieup -tiedown]] [-inst instName -port portName -concat concatList] where if map is specified, use the connection map information to create port/net; otherwise, use the default name to make the connection. The connection format is as follows: "default_net_name [fully_sepecified_signal_list]" where default_net_name refers to the default name for new signals, and fully_sepecified_signal_list refers to a list of fully_specified_signal separated by space. For example: -map "eco_n_1" -map "n781 system.i_cpu.i_ALUB.n781 Outputsystem.i_cpu.i_CCU.n781 Input" . The fully_sepecified_signal format is: signal-full-name [port-direction-value] where port-direction-value is one of (InOut, Input and Output). For example: system.i_cpu.i_ALUB.eco_n_1 InOut Alternatively, \" (backslash and double quote) may be used to quote the default_net_name and fully_specified_signal. By default, the nECO module adds \" to quote both default_net_name and fully_specified_signal. For example: -map \"eco_n_2\" \"system.i_cpu.i_CCU.n1 Input\" \"system.i_cpu.n3\" \"system.i_cpu.i_ALUB.n2 Input\" "</pre>	Connects an instance port to a net.
<pre>_ecoMakeConnection -instport system.i_cpu.eco_i_6 B system.i_cpu.i_PCU.eco_i_7 Z ecoMakeConnection -instport "system.i_cpu1.i_ALUB.eco_i_2" "CO" -signal "system.i_cpu1.i_ALUB.IXR[5]" ecoMakeConnection -instport "system.i_cpu.i_ALUB.spareeco_i_2" "B" -tieup ecoMakeConnection -map "\"eco_n_1\" \"system.i_cpu.i_CCU.eco_p1 Input\" \"system.i_cpu.i_ALUB.eco_p1 Input\"" -instport "system.i_cpu.i_CCU.eco_i_2" "A" "system.i_cpu.i_ALUB.eco_i_1" "B" ecoMakeConnection -inst "system.i_cpu.i_CCU" -port "data" -concat "n1,n2,n3,n4,n5,n6,n7,n8"</pre>	

<code>_ecoRenameNet -old oldNetName -new newNetName [-delim delim]</code> Limitation: only apply to newly created nets	Renames the net statement. Note: A warning message is issued if instance does not exist.
<code>_ecoRenameNet -old "TOP.ff" -new "TOP.z4"</code>	
<code>_ecoRenameInst -inst instName -new newName [-delim delim]</code>	Renames the instance. Note: A warning message is issued if instance does not exist.
<code>_ecoRenameInst -inst "system.i_cpu.i_ALUB.eco_i_3" -new "new"</code>	
<code>_ecoCloneModule instance-cell_pair_list</code>	Clones the module. Note: A warning message is issued if instance for <code>ecoCloneModule</code> does not exist.
<code>_ecoCloneModule {i_cpu2,CPU_2} //clone the module of instance "i_cpu2" to "CPU_2"</code>	
<code>_ecoChangeInstScope -scope scope_name -inst inst_name -new new_instance_name</code> -scope scope_name - Specifies the new destination scope name where the instance must be relocated -inst inst_name - Re-locates the instance to the specified scope. -new new_instance_name - Specifies the new instance name of the scope where the instance must be re-located.	Moves an instance to a specified scope and breaks the original connection.
<code>_ecoChangeInstScope -scope "system.i_cpu.i_ALUB" -inst "system.i_cpu.i_CCU.C20_reg" -new "C20_reg"</code>	
<code>_ecoBuffer -master buffer_master_cell_name -inst buffer_inst_name -old "old_design_net" "new_buffer_inst_port" -new "new_buffer_net" "buff_inst_port"</code> -inst buffer_inst_name - Specifies the instance buffer name which needs to be added -master buffer_master_cell_name - Specifies the master buffer cell name -old "old_design_net" "new_buffer_inst_port" - Specifies the newly added buffer instance port that must be connected to the design net -new "new_buffer_net" "buff_inst_port" "instPort1" "instPort2"... "instPortN" - Specifies the existing instance port list that is connected to the old design net and must be connected to the new buffer net.	Adds the buffer instance to a net.

<pre>_ecoBuffer -master "IVP" -inst "system.i_cpu.i_ALUB.U243" -old "system.i_cpu.i_ALUB.reset" "system.i_cpu.i_ALUB.U243.A" -new "system.i_cpu.i_ALUB.eco_n3" "system.i_cpu.i_ALUB.U243.Z"</pre>	
<pre>_ecoUnbuffer -inst removed_buffer_inst -signal reserved_signal -removed removed_signal</pre> • -inst removed_buffer_inst - Removes the buffer name which is added • -signal reserved_signal - Removes the buffer and retains the signal • -removed removed_signal - Removes the signal after removing the buffer.	Removes the buffer instance from the net.
<pre>_ecoUnbuffer -inst "system.i_cpu.i_ALUB.U243" -signal "system.i_cpu.i_ALUB.eco_n3" -remove "system.i_cpu.i_ALUB.eco_n1"</pre>	
<pre>_ecoCreatePort -module module_name -port port_name -direction input output inout</pre> • -module module_name - Specifies the module name. • -port port_name - Specifies the port name. • -direction input output inout - Specifies the directions. The directions include: input, output, or inout.	Creates a port after you enter the Module Name , the Port Name , and the Port Direction .
<pre>ecoCreatePort -module "CCU" -port "port1" -direction input</pre>	
<pre>_ecoCloneLogic -scope full_hier_name -inst inst_short_name -prefix -retainInput -retainOutput</pre> • -scope "full_hier_name - Specifies the complete hierarchical name of the cell that needs to be cloned. • -inst "inst_short_name" • [-prefix] [-retainInput] [-retainOutput]	Clones a new cell from the existing cell.
<pre>_ecoCloneLogic -scope "system.i_cpu.i_CCU" -inst "\next_MA_reg\7]" -prefix -retainInput -retainOutput</pre>	
<pre>ECOSessionBegin [-comment comment]</pre>	Begins editing session commands (only commands with prefix _eco are allowed to be included in the begin/end session).
<pre>ECOSessionBegin -comment "synopsys 2013.01.08"</pre>	

ECOSessionEnd	Ends editing session commands.
<i>ECOSessionEnd</i>	Commits the session changes to netlist.
ECOChangeInstScope -scope destScopeName -inst instName [-comment comment]	Changes instance scope and commit.
<i>ECOChangeInstScope -scope "system.CHILD1" -inst "system.CHILD1.SubCHILD1.a" -comment "Novas ECO updated by shelley Jul 6, 2001 18:16 "</i>	
ECOCloneModule [-comment comment] instance-cell_pair_list	Clones module and commit.
<i>ECOCloneModule {i_cpu2,CPU_2}</i>	
ECOConfirmSpareCellList	Confirms the spare cell number list setting.
<i>ECOConfirmSpareCellList -file ./SpareCell.txt</i>	
ECOSaveSpareCellList -file file_name	Saves the file name that contains the spare cell list.
<i>ECOSaveSpareCellList -file ./SpareCell.txt</i>	
ECOLoadSpareCellList -file file_name	Loads the file name that contains the spare cell list.
<i>ECOLoadSpareCellList -file ./SpareCell.txt</i>	
ECOConfirmSpareCellNum	Confirms the spare cell number setting.
<i>ECOConfirmSpareCellNum</i>	
ECOSaveSpareCellNum -file file_name	Saves the number list to a file for further use of the spare cell functions.
<i>ECOSaveSpareCellNum -file spareCell.txt</i>	
ECOLoadSpareCellNum -file file_name	Loads the spare cell list with instance name from the file.
<i>ECOLoadSpareCellNum -file spareCell.txt</i>	
ECOLoadSpareCell spare_cell_file	If there is a spare-cell list, use ECOLoadSpareCell to load it.
<i>ECOSaveSpareCell spare_cell_file</i>	If there is a spare-cell list, use ECOSaveSpareCell to save it.

ECOSaveSpareCellReport -file file_name	If there is a spare-cell list, use ECOSaveSpareCellReport to save the report.
<i>ECOSaveSpareCellReport --file ./SpareCell.txt</i>	
ECOSaveDesign [-affect -scope scopename] [-savepath path -orig [prefix]]	Save the modified design. The two options "-orig" and "-savepath" cannot be specified at the same time.
ECOCreatePort -module module_name -port port_name -direction input output inout -comment comments • -module module_name - Specifies the module name • -port port_name - Specifies the port name. • -direction input output inout - Specifies the direction as input, output, or inout • -comment comments - Specifies the comment for the command.	If the ecoCreatePort command is executed in the <i>nSchema</i> window instead of <i>nECO</i> , then this command is logged.
<i>ECOCreatePort -module "system" -port "port1" -direction input -comment "Novas ECO updated by synopsys"</i>	
ECOREplaceCellType -old "old_cell_name" -new "new_cell_name" -addCmt -keynote "comments string" • -old "old_cell_name" - Specifies the old cell name • -new "new_cell_name" - Specifies the new cell name that needs to replace the old cell name. • -addCmt -keynote "comments string" - Specifies the comment for the command	Replaces the old cell name with the new cell name.
<i>ECOREplaceCellType -old "FD2" -new "FD2P" -addCmt -keynote "Novas ECO updated by synopsys"</i>	
ECOChangePortDir portFullHierName input output inout	Changes the port direction list reported by <i>nECO</i> window after <i>nECO</i> commits. Note: A warning message is issued if the change port directory does not exist.
<i>ECOChangePortDir {top.a.port1,input}</i>	
ECODEletePort -port portFullHierName [-comment "comments string"] //comments	Deletes the complete name of the ports. Note: A warning message is issued if the port directory does not exist.
<i>ECODEletePort -port "system.i_cpu.i_CCU.port1" -comment "Novas ECO updated by synopsys"</i>	

ECOSetPref -freezeSilicon on/off -manageMode cellNumber instName -freezeSilicon on/off - Click Tools -> Preferences -> Schematics folder -> ECO folder -> Freeze Silicon page to enable the Freeze Silicon option. -manageMode cellNumber instName - Click Tools -> Preferences -> Schematics folder -> ECO folder -> Freeze Silicon page to enable the spare cell management mode. [-cellQuantityControl] - Specifies whether to control the spare cell by number. This option can be enabled only if the Manage Spare Cells by Instance Name option is enabled.	Sets the preferences for the <i>nECO</i> window.
ECOSetPref -freezeSilicon on -manageMode cellNumber	
ECOSearchSpareCell -nameRule "searching string"	Search matches the instances and adds it to the spare cell form. The search string supports wild card characters.
ECOSearchSpareCell -nameRule system.i_cpu.i_ALUB.*	

Loading the ECO Script Through Batch Mode

If you already have an ECO script file, that is, *novas.eco*, you can load the ECO script file in the batch mode using the following command:

```
verdi -f run.f -ecoScript novas.eco
```

Note:

A warning message is issued if the Verdi import design fails.

Example

The following example shows how to use Verdi provided Novas ECO script to perform ECO changes.

```
# The original net list is
#   gate U282 is driving gate U253 via net n742
#   gate U284 is driving gate U254 via net n744
#   gate U286 is driving gate U255 via net n746
#
# The following Novas eco script will first perform an
# eco change for putting an inverter between above gates
# After eco, the net list will become
#
# U282 is driving eco_i_1 via net n742 and
```

```
#          eco_i_1 is driving U253 via net eco_n_1
# U284 is driving eco_i_2 via net n744 and
#          eco_i_2 is driving U254 via net eco_n_2
# U286 is driving eco_i_3 via net n746 and
#          eco_i_3 is driving U255 via net eco_n_3
#
# Then replace the gate U236 with cell type YCBUFP
#
ECOSessionBegin -comment "1st updated by Novas eco script"
for {set i 1} {$i < 4} {incr i 1} {
    set net_id [expr 740 + $i * 2]
    set inst_id [expr 252 + $i]
    # add the inverter instance first
    _ecoAddInst -master "IV" -inst "system.i_cpu.i_ALUB.eco_i_$i"
    # connect the input pin of inverter
    _ecoMakeConnection \
        -instport "system.i_cpu.i_ALUB.eco_i_$i" "A" \
        -signal "system.i_cpu.i_ALUB.n$net_id"
    # delete the input connection of U25X
    _ecoDeleteConnection -instport "system.i_cpu.i_ALUB.U$inst_id" "A"
    # connect the output pin of inverter
    _ecoMakeConnection -map "eco_n_$i" \
        -instport "system.i_cpu.i_ALUB.eco_i_$i" "Z" \
        "system.i_cpu.i_ALUB.U$inst_id" "A"
}
# to replace the gate U236 with cell type YCBUFP
_ecoReplaceInst -master "YCBUFP" \
    -inst "system.i_cpu.i_CCU.U236"
ECOSessionEnd
#
# The following Novas eco script will perform an eco change for
# moving gate U258 from scope system.i_cpu.i_CCU to scope
# system.i_cpu.i_ALU
#
ECOChangeInstScope -scope "system.i_cpu.i_ALUB" \
    -inst "system.i_cpu.i_CCU.U258" \
    -comment "2nd updated by Novas eco script"
```